

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
CE-5507 — Modelación Hardware Software Orientado a Objetos
I Semestre 2026

LinProd

Documentación Técnica del Proyecto 2

Integrantes

Carlos Eduardo Zaldaña Lima
Maria Laura Iglesias Gutierrez
Yendry Melissa Calderón Gómez
José Andrés Vargas Torres

Fecha de entrega: 16 de mayo de 2026

Índice

1. Diagrama de Clases	2
1.1. Descripción de las clases principales	2
2. Alternativas de Solución Consideradas	3
2.1. Framework de interfaz gráfica	3
2.2. Arquitectura del motor de simulación	4
2.3. Orden de tick dentro de un proceso	4
2.4. Librería de generación de PDF	5
3. Problemas Conocidos	5
4. Problemas Encontrados	5
4.1. Problema 1: Doble-tick de productos dentro de un proceso	5
4.2. Problema 2: Error <code>ModuleNotFoundError</code> al ejecutar scripts directamente . .	7
4.3. Problema 3: Soporte de UTF-8 en reportlab	7
4.4. Problema 4: <code>sim.run()</code> requería argumento obligatorio	8
5. Conclusiones y Recomendaciones	9
5.1. Conclusiones	9
5.2. Recomendaciones	9
6. Bibliografía	10

Diagrama de Clases

La Figura 1 muestra el diagrama UML de clases del sistema LinProd. El sistema está organizado en cuatro módulos con dependencias estrictamente unidireccionales: `model.py` ← `simulator.py` ← {`app.py`, `reports.py`}.

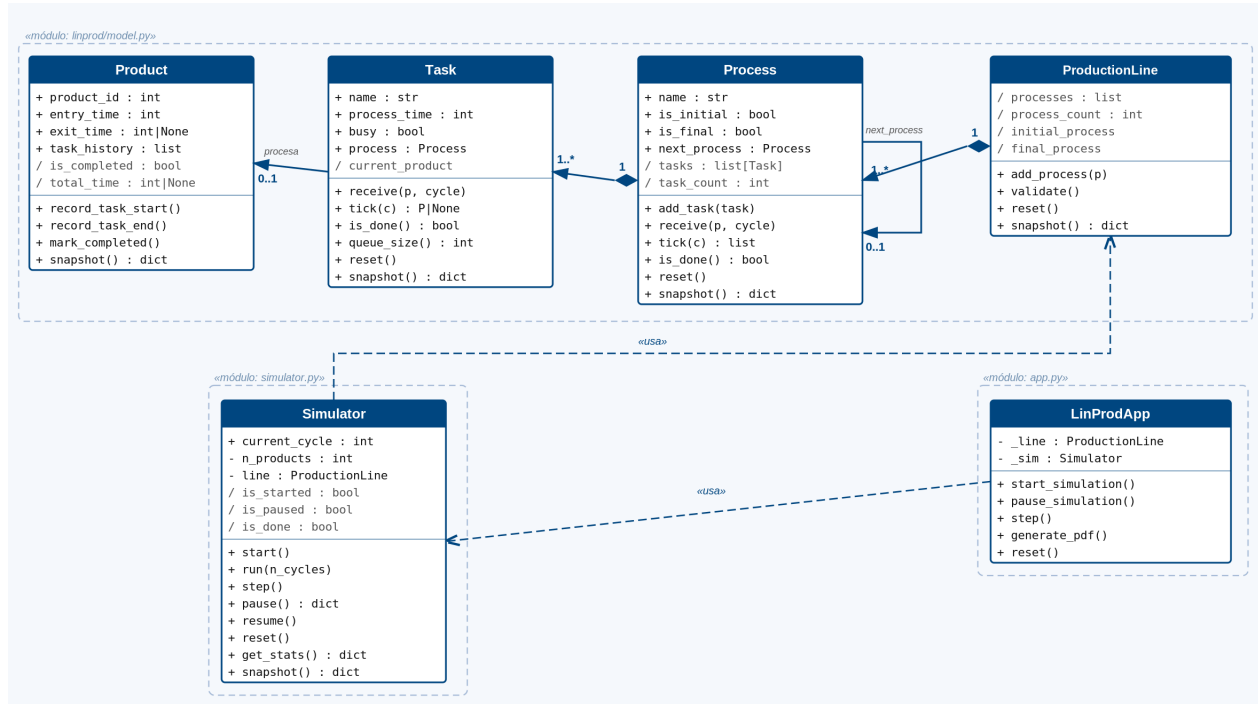


Figura 1: Diagrama de clases del sistema LinProd. Notación: diamante relleno = composición, flecha sólida = asociación, flecha punteada = dependencia (`«usa»`). Los atributos con / son propiedades derivadas de solo lectura.

Descripción de las clases principales

Product Representa un producto que recorre la línea. Almacena su historial de paso por cada tarea (`task_history`) y calcula el tiempo total de forma derivada.

Task Modela una máquina con tiempo de procesamiento fijo. Atiende un producto a la vez; los demás esperan en una cola FIFO (`deque`). Lleva estadísticas de espera promedio usadas por el reporteador.

Process Agrupa una lista ordenada de **Task**. Los productos atraviesan las tareas secuencialmente antes de salir al siguiente proceso. Mantiene referencia al proceso siguiente (`next_process`).

ProductionLine Cadena de **Process** validada: exactamente un proceso marcado como `is_initial` y uno como `is_final`, cada uno con al menos una tarea.

Simulator Motor de eventos discretos. Inyecta productos en el proceso inicial, avanza ciclo a ciclo y gestiona las transferencias entre procesos con semántica de *pipeline real*: un producto que termina el proceso N en el ciclo K no entra al proceso $N + 1$ hasta el ciclo $K + 1$.

LinProdApp Interfaz gráfica Tkinter (subclase de `tk.Tk`). Construye la **ProductionLine** a partir de la entrada del usuario y controla el **Simulator**. Incluye el botón de generación de PDF.

Alternativas de Solución Consideradas

Framework de interfaz gráfica

Alternativa	Ventajas	Desventajas	Selección
Tkinter	Incluido en la stdlib de Python, sin instalación adicional. Documentación amplia. Suficiente para el alcance del proyecto.	Apariencia anticuada por defecto; requiere estilos manuales para un diseño moderno.	✓ Elegida
PyQt6 / PySide6	Widgets más modernos, mayor control visual, Qt Designer.	Requiere <code>pip install pyqt6</code> . Licencia LGPL. Curva de aprendizaje mayor.	Descartada
wxPython	Apariencia nativa del sistema operativo.	Documentación más escasa, instalación más compleja en Linux.	Descartada

Justificación: Tkinter fue elegida por ser parte de la distribución estándar de Python, eliminando la necesidad de dependencias externas para la GUI. El diseño visual se modernizó mediante widgets personalizados (`ModernButton`, `ModernEntry`, `Card`) con un tema oscuro implementado manualmente.

Arquitectura del motor de simulación

Alternativa	Ventajas	Desventajas	Selección
OOP ciclo-a-ciclo	Mapeo natural al dominio (productos, tareas, procesos como objetos). Fácil de depurar ciclo a ciclo. Determinismo garantizado.	Menos eficiente que una cola de eventos para sistemas con muchos eventos dispersos.	✓ Elegida
Cola de eventos (heapq)	Mayor eficiencia en sistemas grandes (salta ciclos vacíos). Estándar en simuladores industriales.	Complejidad adicional; difícil de implementar correctamente en el tiempo disponible.	Descartada
Loop imperativo	Implementación simple.	Difícil de mantener; viola el principio OOP del enunciado.	Descartada

Justificación: El enfoque OOP ciclo-a-ciclo se seleccionó porque mapea directamente al enunciado del proyecto (el tiempo se mide en ciclos t) y facilita la generación del snapshot de estado en cualquier ciclo, requerido para la funcionalidad de pausa.

Orden de tick dentro de un proceso

Este es el diseño más sutil del proyecto. Dentro de `Simulator._advance_one_cycle()`, las tareas de cada proceso se iteran en orden inverso (última $\rightarrow$ primera).

Orden	Comportamiento	Problema	Selección
Inverso (última $\rightarrow$ primera)	Una tarea que entrega su producto a la siguiente ya no es ticked de nuevo ese ciclo (la siguiente ya se procesó). Semántica de pipeline correcta.	Contra-intuitivo; requiere comentario explicativo.	✓ Elegida
Directo (primera $\rightarrow$ última)	Orden natural.	Cuando T1 termina y entrega a T2, T2 es ticked inmediatamente en la misma iteración. El producto consume un ciclo de T2 en el mismo ciclo en que llegó: <i>dobles-ticks</i> .	Descartada

Justificación: Con tick inverso, T2 ya ha sido procesada cuando T1 le entrega el producto, por lo que el producto recién recibido espera hasta el ciclo siguiente. Esto reproduce la semántica real de una línea de montaje y produce los valores de referencia correctos: Producto 1 = ciclo 12, Producto 2 = ciclo 16, Producto 3 = ciclo 20 (verificado con 65 pruebas).

Librería de generación de PDF

Alternativa	Ventajas	Desventajas	Selección
reportlab	Control total sobre el diseño. Mencionada explícitamente en la guía del proyecto. API estable y madura.	La fuente por defecto (Helvetica) no soporta UTF-8; requiere registrar DejaVu.	✓ Elegida
fpdf2	API más simple. Soporte UTF-8 nativo.	Menos control sobre tablas complejas.	Descartada
pdfkit / weasyprint	Genera PDF desde HTML/CSS.	Requiere binario externo (<code>wkhtmltopdf</code>). Mayor complejidad de instalación.	Descartada

Problemas Conocidos

Al momento de la entrega no se conocen problemas abiertos en el sistema. Las 65 pruebas automatizadas pasan, la GUI funciona con líneas de hasta N procesos ilimitados, y el reporte PDF se genera correctamente para todas las configuraciones probadas.

Si el sistema no tiene la fuente DejaVu instalada en `/usr/share/fonts/truetype/dejavu/`, el reporte PDF usa Helvetica como respaldo, lo que puede causar que los caracteres con tilde (, , , etc.) no se muestren en el PDF. El programa *no falla*; simplemente degrada la tipografía. Solución: instalar las fuentes con `sudo apt-get install fonts-dejavu`.

Problemas Encontrados

Problema 1: Doble-tick de productos dentro de un proceso

Descripción

Durante el desarrollo del motor de simulación (`simulator.py`), se detectó que los productos terminaban antes de lo esperado según los valores de referencia del enunciado. Con la línea de referencia (Ensamblado → Pintura → Control), el Producto 1 salía en el ciclo 11 en lugar del ciclo 12 correcto.

Diagnóstico

Al simular con iteración directa (tarea 1 → tarea 2), cuando la Tarea 1 terminaba en el ciclo K y entregaba el producto a la Tarea 2, el bucle inmediatamente ticked a la Tarea 2 en ese mismo ciclo K . El producto recibía un ciclo gratuito de procesamiento, adelantando todos los tiempos de salida en un ciclo.

```
# C digo problem tico (orden directo - primer intento):
for i, task in enumerate(tasks):
    finished = task.tick(current_cycle)
    if finished and i < n_tasks - 1:
        tasks[i+1].receive(finished, current_cycle)
    # tasks[i+1] tambien ser ticked en esta misma iteracion
```

Intentos de solución sin éxito

1. Diferir la entrega un ciclo con lista temporal: se intentó guardar los productos a entregar en una lista `pending_intra` y aplicarlos al inicio del siguiente ciclo. Esto corregía el problema intra-proceso pero rompía la semántica inter-proceso (ahora había dos ciclos de retardo en lugar de cero intra y uno inter).
2. Marcar tareas como «recién cargadas»: se añadió una bandera `just_received` en `Task.receive()` para que `tick()` no procesara en el mismo ciclo. Funcionó pero introdujo estado compartido de difícil mantenimiento y falló con colas.

Solución encontrada

Iterar las tareas en orden inverso dentro de cada proceso:

```
# Solucion: orden inverso (ltima primera tarea)
for i in range(n_tasks - 1, -1, -1):
    task = tasks[i]
    finished = task.tick(self.current_cycle)
    if finished is not None:
        if i == n_tasks - 1:
            # sali del proceso completo
            new_pending.append((finished, proc.next_process))
        else:
            # entrega a la siguiente tarea
            tasks[i + 1].receive(finished, self.current_cycle)
    # tasks[i+1] ya fue ticked antes en esta iteracion
```

Al tickear de último a primero, cuando la Tarea 1 (índice 0) entrega a la Tarea 2 (índice 1), la Tarea 2 ya terminó su tick en esta iteración y no recibirá otro hasta el siguiente ciclo.

Verificación

La prueba de referencia confirma la solución:

```
assert by_id[1]["exit_time"] == 12    # Producto 1
assert by_id[2]["exit_time"] == 16    # Producto 2
assert by_id[3]["exit_time"] == 20    # Producto 3
```

Recomendación

Al diseñar simuladores de pipeline ciclo-a-ciclo, el orden de tick de los elementos es un invariante no trivial. Se recomienda documentarlo con un comentario en el código y cubrirlo explícitamente con una prueba (ver `test_reverse_tick_order_within_process`).

Problema 2: Error `ModuleNotFoundError` al ejecutar scripts directamente

Descripción

Al ejecutar `python3 linprod/simulator.py` desde la raíz del repositorio, Python lanzaba `ModuleNotFoundError: No module named 'linprod'` porque los imports relativos (`from linprod.model import ...`) no resuelven correctamente cuando el script se ejecuta como archivo plano.

Solución encontrada

Ejecutar los módulos como paquete usando la bandera `-m`:

```
python3 -m linprod.simulator    # correcto
python3 linprod/simulator.py    # falla
```

Los comandos de prueba del proyecto (incluyendo `python3 -m linprod.app` para la GUI y `python3 -m linprod.reports` para el reporte) usan consistentemente esta convención.

Problema 3: Soporte de UTF-8 en reportlab

Descripción

Al generar el reporte PDF con la fuente por defecto `Helvetica`, los caracteres con tilde y la letra ñ aparecían como signos de interrogación o caracteres corruptos en el PDF.

Diagnóstico

La fuente Helvetica (fuente Type 1 integrada en reportlab) no incluye el mapa de caracteres extendido de Latin-1/UTF-8. Cualquier carácter fuera del ASCII básico falla silenciosamente.

Solución encontrada

Registrar la fuente TrueType DejaVuSans antes de construir el documento:

```
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont

_DEJAVU_PATHS = [
    "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf",
    "/usr/share/fonts/dejavu/DejaVuSans.ttf",
]

def _register_fonts():
    for path in _DEJAVU_PATHS:
        if os.path.exists(path):
            pdfmetrics.registerFont(TTFont("DejaVu", path))
            return "DejaVu"
    return "Helvetica" # fallback
```

Recomendación

Siempre registrar una fuente TTF con soporte Unicode en proyectos reportlab que usen español. La ruta más portátil en sistemas Debian/Ubuntu es `/usr/share/fonts/truetype/dejavu/DejaVuSans`.

Problema 4: `sim.run()` requería argumento obligatorio

Descripción

La guía del Módulo 2 mostraba el uso de `sim.run()` sin argumentos para «correr hasta terminar», pero la firma original era `def run(self, n_cycles: int)`, lo que hacía `n_cycles` obligatorio y lanzaba `TypeError` al omitirlo.

Solución

Se añadió un valor por defecto de 100 000 ciclos:

```
def run(self, n_cycles: int = 100_000) -> None:
    ...
    for _ in range(n_cycles):
        if self._paused or self.is_done:
            break
        self._advance_one_cycle()
```

El loop rompe antes de llegar al límite si `is_done` se activa, por lo que en la práctica `run()` sin argumento equivale a «correr hasta que todos los productos completen la línea».

Conclusiones y Recomendaciones

Conclusiones

1. El paradigma OOP es adecuado para simulación de líneas de producción. La correspondencia directa entre los objetos del dominio (productos, tareas, procesos) y las clases Python redujo la brecha entre el modelo conceptual y la implementación, facilitando el diseño, la depuración y las pruebas.
2. Los invariantes sutiles del motor de simulación deben documentarse y probarse explícitamente. El orden inverso de tick es un detalle no obvio que, si se rompe, produce resultados incorrectos sin errores visibles. La prueba `test_reverse_tick_order_within_process` es la salvaguarda crítica.
3. La cobertura de pruebas (65 pruebas) brinda confianza en el determinismo. La prueba de referencia contra los valores del enunciado ($P1=12$, $P2=16$, $P3=20$) actúa como *golden test*: cualquier regresión en la lógica central lo romperá primero.
4. La separación estricta de módulos (`model` $\leftarrow$ `simulator` $\leftarrow$ `app/reports`) facilita el trabajo en paralelo. Cada módulo pudo desarrollarse y probarse de forma independiente, reduciendo conflictos de integración.
5. Reportlab es potente pero requiere atención al soporte de caracteres. La gestión de fuentes TTF es un paso adicional necesario para producir documentos profesionales en español con tildes y ñ.

Recomendaciones

1. Mantener el invariante de tick inverso documentado. Si en el futuro se necesita soporte para procesos con semántica diferente (e.g., tareas en paralelo dentro de un proceso), rediseñar esa parte sin comprometer el tick inverso de las tareas secuenciales.

2. Extender la suite de pruebas con líneas de 5+ procesos para cubrir el escenario exacto que el profesor presentará en la defensa.
3. Considerar exportar el snapshot de pausa en formato JSON para facilitar la depuración y la revisión post-ejecución.
4. Para producción real, reemplazar el límite de 100 000 ciclos en `run()` por detección explícita de *deadlock* (todos los procesos sin movimiento por N ciclos consecutivos).

Bibliografía

- [1] Instituto Tecnológico de Costa Rica. *Enunciado del Proyecto 2 — LinProd*. CE-5507 Modelación Hardware Software Orientado a Objetos, I Semestre 2026.
- [2] Python Software Foundation. *The Python Standard Library — tkinter: Python interface to Tcl/Tk*. Disponible en: <https://docs.python.org/3/library/tkinter.html> [Acceso: mayo 2026].
- [3] Python Software Foundation. *collections.deque — Double-ended queue*. Disponible en: <https://docs.python.org/3/library/collections.html#collections.deque> [Acceso: mayo 2026].
- [4] ReportLab Inc. *ReportLab PDF Library User Guide*. Disponible en: <https://www.reportlab.com/docs/reportlab-userguide.pdf> [Acceso: mayo 2026].
- [5] Wikipedia. *Producción en cadena*. Disponible en: https://es.wikipedia.org/wiki/Produccion_en_cadena [Acceso: mayo 2026]. (Fuente citada en el marco teórico del enunciado.)
- [6] Wikipedia. *Discrete-event simulation*. Disponible en: https://en.wikipedia.org/wiki/Discrete-event_simulation [Acceso: mayo 2026].
- [7] Law, A. M. *Simulation Modeling and Analysis*, 5th ed. McGraw-Hill, 2015. Cap. 1 y 2: fundamentos de simulación de eventos discretos.
- [8] pytest Development Team. *pytest: helps you write better programs*. Disponible en: <https://docs.pytest.org> [Acceso: mayo 2026].